Building a Nat Geo-Style Researcher Web App with Python Backend

---

✨Goal

To build a Nat Geo-style web app for researchers in the wild to collect, analyze, and visualize data using React (frontend), Tailwind CSS (styling), Firebase (auth/storage), and Python (Flask or FastAPI for backend), with optional integration of Oracle Cloud Infrastructure (OCI) Generative AI APIs.

🌳Phase 1: Project Setup

1. Install Node.js and Python

Ensure Node.js and Python (3.8+) are installed.

- Node.js: https://nodejs.org/
- Python: https://python.org/

2. Create React Frontend

```
npx create-react-app natgeo-research-app
cd natgeo-research-app
```

3. Install Tailwind CSS

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Update `tailwind.config.js`:

```
content: ["./src/**/*.{js,jsx,ts,tsx}"]
```

Update `index.css`:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

4. Setup Python Backend

```
mkdir backend
cd backend
python -m venv venv
source venv/bin/activate  # or venv\Scripts\activate on Windows
pip install fastapi uvicorn python-dotenv firebase-admin
```

Create `main.py` in the `backend` folder:

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to NatGeo Research Backend"}
```

Run the backend server:

```
uvicorn main:app --reload
```

Phase 2: Page Structure

Frontend pages:

- `/login` - Researcher sign-in
- `/dashboard` - Upload and analytics view
- `/profile` - Personal info and notes
- `/observations` - List/map of sightings

Backend endpoints:

- `POST /api/observations`
- `GET /api/observations`
- `POST /api/notes/summarize` (uses OCI GenAI)

🌵 Phase 3: Authentication (Firebase)

1. Setup Firebase Project

- Go to https://console.firebase.google.com
- Enable Email/Password sign-in

2. Install Firebase SDK (Frontend)

```
npm install firebase
```

3. Configure Firebase in React (`firebase.js`):

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT.firebaseapp.com",
  projectId: "YOUR_PROJECT_ID",
};

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
```

4. Firebase Admin SDK (Backend)

```
pip install firebase-admin
```

Use it to verify users and securely store data.

🦌 Phase 4: Data Storage (Firestore)

Frontend:

```
import { getFirestore, collection, addDoc } from "firebase/firestore";
const db = getFirestore(app);
await addDoc(collection(db, "observations"), {...});
```

Backend (Python with `firebase-admin`):

```
import firebase_admin
from firebase_admin import credentials, firestore
cred = credentials.Certificate("path/to/your/serviceAccountKey.json")
firebase_admin.initialize_app(cred)
db = firestore.client()
```

🌍 Phase 5: Maps (React Leaflet)

```
npm install leaflet react-leaflet
```

Display GPS-tagged sightings on a map.

🐃 Phase 6: Image Uploads (Firebase Storage)

Use Firebase Storage SDK in React to upload media files. Store image URLs in Firestore.

Phase 7: AI Integration (OCI Gen AI)

Use Python `requests` to call OCI endpoints and React frontend to submit prompts.

Python example in backend:

```
import requests
headers = {
    "Authorization": "Bearer YOUR_OCI_TOKEN",
    "Content-Type": "application/json"
}
data = {"prompt": "Summarize: ..."}
response = requests.post("https://YOUR_OCI_GEN_AI_URL", headers=headers,
json=data)
result = response.json()
```

React fetch call to FastAPI backend:

```
await fetch("/api/notes/summarize", {
  method: "POST",
```

```
  body: JSON.stringify({ note: noteText })
});
```

Phase 8: Hosting

Frontend Hosting (Firebase)

```
npm install -g firebase-tools
firebase login
firebase init
firebase deploy
```

Backend Hosting (Render.com or Railway.app)

Push Python backend to GitHub and connect to Render or Railway.

Final Notes

- Use Tailwind for elegant and responsive UI
- Backend handles AI logic and secure operations
- Use environment variables for API keys and secrets
- Folder structure:
- `/frontend` - React app
- `/backend` - Python (FastAPI or Flask)

Let me know if you'd like offline mode, PWA, or mobile version.